

Guia Completo do Banco de Dados - Sistema CRM

Índice

1. [Visão Geral do Sistema](#)
 2. [Estrutura Organizacional](#)
 3. [Pessoas e Recursos](#)
 4. [Fluxo Comercial](#)
 5. [Gestão de Projetos](#)
 6. [Catálogos e Configurações](#)
 7. [Segurança e Controle de Acesso](#)
 8. [Conceitos Técnicos Explicados](#)
 9. [Relacionamentos entre Tabelas](#)
 10. [Exemplos Práticos de Uso](#)
-

1. Visão Geral do Sistema

O banco de dados foi projetado para um **Sistema CRM (Customer Relationship Management)** completo, que gerencia:

- **Estrutura da empresa** (coligadas, filiais)
- **Clientes, fornecedores e parceiros**
- **Pessoas** (funcionários, colaboradores)
- **Fluxo comercial** (orçamentos → propostas → contratos)
- **Projetos e tarefas**
- **Controle de acesso** (usuários, perfis, permissões)

Organização em 6 Grupos Funcionais

Plain Text

SISTEMA CRM

1. ESTRUTURA ORGANIZACIONAL	
└ Coligada → Filial → Organizacao (Cliente/Fornecedor)	
2. PESSOAS E RECURSOS	
└ Pessoa → Funções → Habilidades → Contatos	
3. FLUXO COMERCIAL	
└ Orçamento → Proposta → Contrato	
4. GESTÃO DE PROJETOS	
└ Projeto → Fase → Atividade → Tarefa → Pessoas	
5. CATÁLOGOS E CONFIGURAÇÕES	
└ Produtos, Serviços, Status, Módulos	
6. SEGURANÇA E CONTROLE DE ACESSO	
└ Usuario → Perfil → Permissões	

2. Estrutura Organizacional

2.1 Tabela: Coligada

Para que serve: Representa as empresas que você controla no sistema. É o nível mais alto da hierarquia organizacional.

Exemplo prático:

- Coligada 1: "Empresa Matriz Ltda"
- Coligada 2: "Empresa Filial SP Ltda"
- Coligada 0: "Todas as Coligadas" (valor especial para filtros globais)

Campos principais:

- `id` : Identificador único (1, 2, 3...)
- `codigo` : Código da coligada (ex: "COL001")
- `nome` : Nome fantasia (ex: "Empresa Matriz")
- `cnpj` : CNPJ da empresa
- `ativo` : Se está ativa (1) ou inativa (0)

Relacionamentos:

- Uma Coligada pode ter **várias Filiais**

- Uma Coligada pode ter **várias Organizações** (clientes/fornecedores)
 - Uma Coligada pode ter **várias Pessoas** (funcionários)
-

2.2 Tabela: Filial

Para que serve: Representa as unidades operacionais de cada coligada (filiais, escritórios, lojas).

Exemplo prático:

Plain Text

```
Coligada: "Empresa Matriz Ltda"  
├ Filial 1: "Matriz São Paulo"  
├ Filial 2: "Filial Rio de Janeiro"  
└ Filial 3: "Filial Belo Horizonte"
```

Campos principais:

- `id` : Identificador único
- `coligada_id` : A qual coligada pertence (chave estrangeira)
- `codigo` : Código da filial (ex: "FIL-SP")
- `nome` : Nome da filial
- `matriz` : Se é a matriz (1) ou não (0)

Relacionamentos:

- Uma Filial pertence a **uma Coligada**
 - Uma Filial pode ter **várias Pessoas** alocadas
 - Uma Filial pode atender **várias Organizações** (clientes)
-

2.3 Tabela: Organizacao

Para que serve: Representa empresas externas que se relacionam com você (clientes, fornecedores, parceiros, prospects).

Exemplo prático:

Plain Text

```
Organizacao 1:  
- tipo: "CLIENTE"  
- nome_fantasia: "Bevap Tecnologia"
```

- cnpj_cpf: "12.345.678/0001-90"
- coligada_id: 1 (controlada pela Coligada 1)
- filial_id: 2 (atendida pela Filial 2)

Organizacao 2:

- tipo: "FORNECEDOR"
- nome_fantasia: "Fornecedor de Software Ltda"

Campos principais:

- tipo : CLIENTE, FORNECEDOR, PARCEIRO ou PROSPECT
- nome_fantasia : Nome comercial
- razao_social : Nome jurídico
- cnpj_cpf : CNPJ ou CPF
- coligada_id : Qual coligada controla este relacionamento
- filial_id : Qual filial atende (opcional)

Por que unificamos Cliente/Fornecedor/Empresa em uma tabela?

- Evita duplicação de dados
- Uma empresa pode ser cliente E fornecedor ao mesmo tempo
- Facilita manutenção e consultas

Relacionamentos:

- Uma Organização pertence a **uma Coligada**
- Uma Organização pode ser atendida por **uma Filial**
- Uma Organização pode ter **vários Contatos** (gestores)
- Uma Organização pode ter **vários Orçamentos**
- Uma Organização pode ter **vários Projetos**

3. Pessoas e Recursos

3.1 Tabela: Pessoa

Para que serve: Cadastra todas as pessoas físicas do sistema (funcionários, colaboradores, parceiros, usuários).

Exemplo prático:

Bash

Pessoa 1:

- nome: "João Silva"
- cpf: "123.456.789-00"
- email: "joao.silva@empresa.com"
- tipo: "FUNCIONARIO"
- coligada_id: 1
- filial_id: 2

Campos principais:

- tipo : FUNCIONARIO, COLABORADOR, PARCEIRO, USUARIO
- nome : Nome completo
- cpf : CPF único
- email : Email único
- coligada_id : A qual coligada pertence
- filial_id : Em qual filial trabalha (opcional)

Relacionamentos:

- Uma Pessoa pertence a **uma Coligada**
- Uma Pessoa pode trabalhar em **uma Filial**
- Uma Pessoa pode ter **várias Funções** ao longo do tempo
- Uma Pessoa pode ter **várias Habilidades**
- Uma Pessoa pode ser responsável por **várias Tarefas**
- Uma Pessoa pode ter **um Usuário** (login no sistema)

3.2 Tabela: Funcao

Para que serve: Define os cargos/funções disponíveis no sistema.

Exemplo prático:

Plain Text

Funcao 1: "Gerente de Projetos" (nível TATICO)
Funcao 2: "Desenvolvedor Senior" (nível TECNICO)
Funcao 3: "Analista de Negócios" (nível TECNICO)
Funcao 4: "Diretor" (nível ESTRATEGICO)

Campos principais:

- codigo : Código único (ex: "GER-PROJ")

- `nome` : Nome da função
 - `nivel` : OPERACIONAL, TECNICO, TATICO, ESTRATEGICO
-

3.3 Tabela: PessoaFuncao (Associativa)

Para que serve: Relaciona pessoas com suas funções ao longo do tempo. Uma pessoa pode ter múltiplas funções em períodos diferentes.

Exemplo prático:

Plain Text

João Silva:

- De 2020 a 2022: "Desenvolvedor Senior"
- De 2022 até hoje: "Gerente de Projetos" (`principal = 1`)

Campos principais:

- `pessoa_id` : Qual pessoa
- `funcao_id` : Qual função
- `data_inicio` : Quando começou nesta função
- `data_fim` : Quando terminou (NULL = ainda ativa)
- `principal` : Se é a função principal atual (1) ou não (0)

Por que uma tabela separada?

- Permite histórico de funções
 - Uma pessoa pode ter múltiplas funções simultaneamente
 - Facilita relatórios de evolução de carreira
-

3.4 Tabela: Habilidade

Para que serve: Define as competências técnicas ou comportamentais que pessoas podem ter.

Exemplo prático:

Plain Text

Habilidade 1: "Python" (`categoria: TECNICA`)

Habilidade 2: "Liderança" (`categoria: COMPORTAMENTAL`)

Habilidade 3: "Inglês" (categoria: IDIOMA)
Habilidade 4: "PMP" (categoria: CERTIFICACAO)

3.5 Tabela: PessoaHabilidade (Associativa)

Para que serve: Relaciona pessoas com suas habilidades e níveis de proficiência.

Exemplo prático:

Plain Text

```
João Silva:
- Python: AVANÇADO
- JavaScript: INTERMEDIÁRIO
- Liderança: AVANÇADO
- Inglês: ESPECIALISTA
```

Campos principais:

- `peessoa_id` : Qual pessoa
- `habilidade_id` : Qual habilidade
- `nivel` : BÁSICO, INTERMEDIÁRIO, AVANÇADO, ESPECIALISTA
- `data_avaliacao` : Quando foi avaliada
- `avaliado_por` : Quem avaliou (outra pessoa)

3.6 Tabela: Contato

Para que serve: Cadastra contatos de organizações (gestores, responsáveis, tomadores de decisão).

Exemplo prático:

Plain Text

```
Organização: "Bevap Tecnologia"
Contato 1:
- nome: "Maria Santos"
- cargo: "Gerente de TI"
- email: "maria@bevap.com"
- principal: 1 (contato principal)

Contato 2:
- nome: "Pedro Oliveira"
- cargo: "Diretor Financeiro"
```

- email: "pedro@bevap.com"
- principal: 0

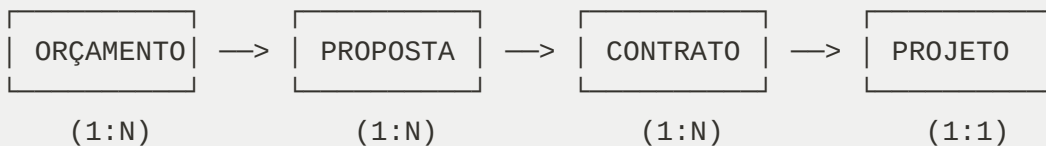
Por que separar Contato de Pessoa?

- Contatos são pessoas externas (de outras empresas)
- Pessoas são internas (seus funcionários)
- Contatos não precisam de usuário no sistema
- Contatos estão vinculados a Organizações

4. Fluxo Comercial

O fluxo comercial segue uma sequência lógica:

Plain Text



4.1 Tabela: Orcamento

Para que serve: Registra orçamentos elaborados para clientes.

Exemplo prático:

Plain Text

```
Orçamento ORÇ-2025-001:  
- Cliente: "Bevap Tecnologia"  
- Data emissão: 01/01/2025  
- Data validade: 31/01/2025  
- Título: "Desenvolvimento de Sistema Web"  
- Valor total: R$ 50.000,00  
- Status: "ENVIADO"  
- Responsável: João Silva
```

Campos principais:

- `numero` : Número único sequencial (ORÇ-2025-001)
- `organizacao_id` : Para qual cliente
- `contato_id` : Contato específico do cliente

- `data_emissao` : Quando foi criado
- `data_validade` : Até quando é válido
- `valor_total` : Valor total do orçamento
- `status_id` : Status atual (RASCUNHO, ENVIADO, APROVADO, etc.)
- `responsavel_id` : Quem elaborou

Relacionamentos:

- Um Orçamento é para **uma Organização** (cliente)
- Um Orçamento pode ter **vários Itens** (produtos/serviços)
- Um Orçamento pode gerar **várias Propostas**

4.2 Tabela: OrcamentoItem

Para que serve: Detalha os itens (produtos ou serviços) de um orçamento.

Exemplo prático:

Plain Text

Orçamento ORÇ-2025-001:

Item 1:

- tipo: SERVICO
- descricao: "Desenvolvimento Backend"
- quantidade: 200 (horas)
- valor_unitario: R\$ 150,00
- valor_total: R\$ 30.000,00

Item 2:

- tipo: SERVICO
- descricao: "Desenvolvimento Frontend"
- quantidade: 120 (horas)
- valor_unitario: R\$ 120,00
- valor_total: R\$ 14.400,00

Item 3:

- tipo: PRODUTO
- descricao: "Licença de Software"
- quantidade: 10 (unidades)
- valor_unitario: R\$ 560,00
- valor_total: R\$ 5.600,00

Total do Orçamento: R\$ 50.000,00

Campos principais:

- `orcamento_id` : A qual orçamento pertence
- `tipo` : PRODUTO ou SERVICO
- `produto_id` : Se for produto (referência à tabela Produto)
- `servico_id` : Se for serviço (referência à tabela Servico)
- `quantidade` : Quantidade
- `valor_unitario` : Preço unitário
- `valor_total` : Quantidade × Valor unitário

Por que tipo PRODUTO ou SERVICO?

- Um item só pode ser UM dos dois (exclusivo)
- Facilita relatórios separados
- Permite regras diferentes para cada tipo

4.3 Tabela: Proposta

Para que serve: Formaliza uma proposta comercial baseada em um orçamento aprovado.

Exemplo prático:

Plain Text

Proposta PROP-2025-001:

- Baseada em: Orçamento ORÇ-2025-001
- Data emissão: 05/01/2025
- Título: "Proposta para Desenvolvimento de Sistema Web"
- Valor total: R\$ 48.000,00 (negociado)
- Condições pagamento: "30% entrada, 70% em 3x"
- Prazo entrega: "90 dias"
- Status: "NEGOCIACAO"

Fluxo:

1. Cliente recebe Orçamento
2. Cliente aprova ou negocia
3. Você cria Proposta formal (pode ajustar valores)
4. Cliente aceita ou recusa Proposta

Relacionamentos:

- Uma Proposta vem de **um Orçamento**

- Um Orçamento pode gerar **várias Propostas** (revisões)
- Uma Proposta pode gerar **vários Contratos** (se houver aditivos)

4.4 Tabela: Contrato

Para que serve: Formaliza o contrato assinado com o cliente.

Exemplo prático:

Plain Text

```
Contrato CONT-2025-001:
- Baseado em: Proposta PROP-2025-001
- Data assinatura: 10/01/2025
- Data início: 15/01/2025
- Data fim: 15/04/2025 (90 dias)
- Valor total: R$ 48.000,00
- Status: "VIGENTE"
- Versão: 1
- Arquivo: "contrato_bevasp_2025.pdf"
```

Campos especiais:

- `versao` : Número da versão (para aditivos contratuais)
- `contrato_anterior_id` : Se for aditivo, referência ao contrato anterior
- `arquivo_url` : Link para o PDF do contrato assinado

Relacionamentos:

- Um Contrato vem de **uma Proposta**
- Um Contrato pode gerar **vários Projetos**
- Um Contrato pode ter **várias Versões** (aditivos)

5. Gestão de Projetos

A estrutura de projetos é hierárquica:

Plain Text

```
PROJETO
├── FASE (Iniciação, Planejamento, Execução, etc.)
│   └── ATIVIDADE (Agrupamento de tarefas)
```

- └ TAREFA (Unidade mínima de trabalho)
- └ PESSOAS (Responsáveis, colaboradores)

5.1 Tabela: Projeto

Para que serve: Gerencia projetos executados para clientes.

Exemplo prático:

Plain Text

```
Projeto PROJ-2025-001:
- Nome: "Sistema Web Bevap"
- Cliente: "Bevap Tecnologia"
- Contrato: CONT-2025-001
- Data início: 15/01/2025
- Data fim prevista: 15/04/2025
- Valor orçado: R$ 48.000,00
- Percentual conclusão: 35%
- Status: "EM_EXECUCAO"
- Prioridade: "ALTA"
- Gerente: João Silva
```

Campos principais:

- `codigo` : Código único (PROJ-2025-001)
- `nome` : Nome do projeto
- `organizacao_id` : Cliente
- `contrato_id` : Contrato que originou (opcional)
- `data_inicio` : Quando começou
- `data_fim_prevista` : Previsão de término
- `percentual_conclusao` : 0 a 100%
- `prioridade` : BAIXA, MEDIA, ALTA, URGENTE
- `gerente_id` : Gerente do projeto

Relacionamentos:

- Um Projeto é para **uma Organização** (cliente)
- Um Projeto pode vir de **um Contrato**
- Um Projeto pode ter **várias Fases**
- Um Projeto pode ter **várias Atividades**

5.2 Tabela: Fase

Para que serve: Define fases padrão de projetos (templates reutilizáveis).

Exemplo prático:

Plain Text

```
Fase 1: "Iniciação" (5 dias padrão)
Fase 2: "Planejamento" (10 dias padrão)
Fase 3: "Execução" (30 dias padrão)
Fase 4: "Monitoramento" (durante toda execução)
Fase 5: "Encerramento" (5 dias padrão)
```

Por que separar Fase?

- Reutilização: mesmas fases em vários projetos
- Padronização: todos os projetos seguem mesma estrutura
- Facilita estimativas de prazo

5.3 Tabela: ProjetoFase (Associativa)

Para que serve: Relaciona projetos com suas fases específicas, com datas e status reais.

Exemplo prático:

Plain Text

```
Projeto PROJ-2025-001:
  Fase 1: "Iniciação"
    - Data início: 15/01/2025
    - Data fim prevista: 20/01/2025
    - Data fim real: 19/01/2025
    - Percentual conclusão: 100%
    - Status: "CONCLUIDA"

  Fase 2: "Planejamento"
    - Data início: 20/01/2025
    - Data fim prevista: 30/01/2025
    - Percentual conclusão: 100%
    - Status: "CONCLUIDA"

  Fase 3: "Execução"
    - Data início: 30/01/2025
    - Data fim prevista: 01/03/2025
```

- Percentual conclusão: 45%
- Status: "EM_ANDAMENTO"

Por que uma tabela separada?

- Cada projeto tem suas próprias datas
- Permite acompanhar progresso por fase
- Fases podem ter ordem diferente em cada projeto

5.4 Tabela: Atividade

Para que serve: Agrupa tarefas relacionadas dentro de uma fase.

Exemplo prático:

Plain Text

```
Projeto: "Sistema Web Bevap"
Fase: "Execução"
  Atividade 1: "Desenvolvimento do Backend"
    - Código: AT-001
    - Horas estimadas: 200h
    - Horas realizadas: 85h
    - Percentual conclusão: 42%
    - Status: "EM_ANDAMENTO"
    - Responsável: João Silva

  Atividade 2: "Desenvolvimento do Frontend"
    - Código: AT-002
    - Horas estimadas: 120h
    - Horas realizadas: 30h
    - Percentual conclusão: 25%
    - Status: "EM_ANDAMENTO"
    - Responsável: Maria Santos
```

Por que Atividade entre Fase e Tarefa?

- Organização: agrupa tarefas relacionadas
- Facilita delegação: cada atividade tem um responsável
- Melhora visualização: não fica tudo solto

5.5 Tabela: Tarefa

Para que serve: Unidade mínima de trabalho a ser executada.

Exemplo prático:

Plain Text

Atividade: "Desenvolvimento do Backend"

Tarefa 1: "Criar modelo de dados"

- Código: TAR-001
- Data início: 30/01/2025
- Data fim prevista: 05/02/2025
- Horas estimadas: 20h
- Horas realizadas: 18h
- Prioridade: ALTA
- Percentual conclusão: 100%
- Status: "CONCLUIDA"

Tarefa 2: "Implementar API de autenticação"

- Código: TAR-002
- Depende de: TAR-001 (predecessora)
- Data início: 05/02/2025
- Data fim prevista: 10/02/2025
- Horas estimadas: 30h
- Horas realizadas: 15h
- Prioridade: ALTA
- Percentual conclusão: 50%
- Status: "EM_ANDAMENTO"

Campos especiais:

- `tarefa_predecessora_id` : Tarefa que deve ser concluída antes
- `prioridade` : BAIXA, MEDIA, ALTA, URGENTE
- `ordem` : Ordem de execução

Dependências entre tarefas:

- Tarefa 2 só pode começar depois da Tarefa 1
- Permite criar cronograma realista
- Facilita identificar caminho crítico

5.6 Tabela: TarefaPessoa (Associativa)

Para que serve: Relaciona tarefas com pessoas responsáveis, com papéis específicos.

Exemplo prático:

Plain Text

Tarefa: "Implementar API de autenticação"

Pessoa 1: João Silva

- Papel: RESPONSÁVEL
- Horas estimadas: 20h
- Horas realizadas: 10h

Pessoa 2: Maria Santos

- Papel: COLABORADOR
- Horas estimadas: 10h
- Horas realizadas: 5h

Pessoa 3: Pedro Oliveira

- Papel: REVISOR
- Horas estimadas: 5h
- Horas realizadas: 0h

Papéis disponíveis:

- **RESPONSÁVEL:** Quem é o dono da tarefa
- **COLABORADOR:** Quem ajuda na execução
- **REVISOR:** Quem revisa o trabalho
- **APROVADOR:** Quem aprova a conclusão

Por que separar horas por pessoa?

- Permite apontamento individual
- Facilita relatórios de produtividade
- Ajuda no cálculo de custos

6. Catálogos e Configurações

6.1 Tabela: Produto

Para que serve: Cadastra produtos comercializados.

Exemplo prático:

Plain Text

Produto 1:

- código: "PROD-001"
- nome: "Licença Microsoft Office"
- categoria: "Software"
- unidade: "Unidade"

- valor_unitario: R\$ 560,00
- estoque_atual: 50

6.2 Tabela: Servico

Para que serve: Cadastra serviços prestados.

Exemplo prático:

Plain Text

Serviço 1:

- codigo: "SERV-001"
- nome: "Desenvolvimento Backend"
- categoria: "Desenvolvimento"
- unidade: "Hora"
- valor_hora: R\$ 150,00
- duracao_padrao_horas: 1

6.3 Tabela: Status

Para que serve: Define status que podem ser usados em várias entidades do sistema.

Por que campo "contexto"?

- Status de PROJETO é diferente de status de TAREFA
- Evita misturar status de contextos diferentes
- Facilita filtros e validações

Exemplo prático:

Plain Text

Status de PROJETO:

- PLANEJAMENTO
- EM_EXECUCAO
- PAUSADO
- CONCLUIDO
- CANCELADO

Status de TAREFA:

- PENDENTE
- EM_ANDAMENTO
- EM_REVISAO
- CONCLUIDA

- CANCELADA

Status de ORCAMENTO:

- RASCUNHO
- ENVIADO
- EM_ANALISE
- APROVADO
- REJEITADO

Campo "final":

- Indica se é um status terminal (TRUE)
- Status finais: CONCLUIDO, CANCELADO, APROVADO, REJEITADO
- Útil para relatórios e dashboards

6.4 Tabela: Modulo

Para que serve: Define módulos do sistema (CRM, Financeiro, RH, etc.).

Exemplo prático:

Plain Text

Módulo 1: "CRM" (Customer Relationship Management)
Módulo 2: "Financeiro" (futuro)
Módulo 3: "RH" (futuro)

Por que módulos?

- Preparado para expansão
- Permissões são por módulo
- Facilita licenciamento

6.5 Tabela: Deflator

Para que serve: Registra índices econômicos para correção monetária.

Exemplo prático:

Plain Text

Deflator 1:

- codigo: "IPCA"
- nome: "Índice de Preços ao Consumidor Amplo"
- tipo: "INFLACAO"

- fonte: "IBGE"

Deflator 2:

- codigo: "DOLAR"
- nome: "Dólar Americano"
- tipo: "CAMBIO"
- fonte: "Banco Central"

6.6 Tabela: DeflatorValor

Para que serve: Armazena valores históricos dos deflatores.

Exemplo prático:

Plain Text

IPCA:

- Jan/2025: 0,42%
- Fev/2025: 0,38%
- Mar/2025: 0,45%

DOLAR:

- 01/01/2025: R\$ 5,85
- 01/02/2025: R\$ 5,92
- 01/03/2025: R\$ 5,78

Uso prático:

- Reajuste de contratos
- Correção de valores históricos
- Relatórios financeiros

7. Segurança e Controle de Acesso

7.1 Tabela: Usuario

Para que serve: Cadastra usuários que podem fazer login no sistema.

Exemplo prático:

Plain Text

Usuário 1:

- pessoa_id: 1 (João Silva)

```
- username: "joao.silva"
- email: "joao.silva@empresa.com"
- senha_hash: "$2a$10$..." (criptografada)
- ativo: 1
- bloqueado: 0
- tentativas_login: 0
```

Campos de segurança:

- `senha_hash` : Senha NUNCA é armazenada em texto puro
- `tentativas_login` : Contador de tentativas falhas
- `bloqueado` : Se passou de X tentativas, bloqueia
- `token_recuperacao` : Para recuperação de senha

Relacionamento:

- Um Usuário está vinculado a **uma Pessoa**
- Um Usuário pode ter **vários Perfis**

7.2 Tabela: Perfil

Para que serve: Define perfis de acesso (roles) no sistema.

Exemplo prático:

Plain Text

Perfil 1: "Administrador CRM"

- `modulo_id`: 1 (CRM)
- `nivel`: SUPER_ADMIN
- Permissões: TODAS

Perfil 2: "Gerente de Projetos"

- `modulo_id`: 1 (CRM)
- `nivel`: ADMIN
- Permissões: Gerenciar projetos, tarefas, pessoas

Perfil 3: "Usuário Padrão"

- `modulo_id`: 1 (CRM)
- `nivel`: ESCRITA
- Permissões: Visualizar e editar suas próprias tarefas

Níveis de acesso:

- **LEITURA**: Só visualiza

- **ESCRITA:** Visualiza e edita
 - **ADMIN:** Controle total do módulo
 - **SUPER_ADMIN:** Controle total do sistema
-

7.3 Tabela: Permissao

Para que serve: Define permissões específicas no sistema.

Exemplo prático:

Plain Text

Permissão 1:

- codigo: "CRM_PROJETO_CREATE"
- nome: "Criar Projeto"
- modulo_id: 1 (CRM)
- recurso: "projeto"
- acao: "CREATE"

Permissão 2:

- codigo: "CRM_PROJETO_READ"
- nome: "Visualizar Projeto"
- recurso: "projeto"
- acao: "READ"

Permissão 3:

- codigo: "CRM_PROJETO_UPDATE"
- nome: "Editar Projeto"
- recurso: "projeto"
- acao: "UPDATE"

Permissão 4:

- codigo: "CRM_PROJETO_DELETE"
- nome: "Excluir Projeto"
- recurso: "projeto"
- acao: "DELETE"

Modelo CRUD + Execute:

- **CREATE:** Criar novos registros
- **READ:** Visualizar registros
- **UPDATE:** Editar registros
- **DELETE:** Excluir registros
- **EXECUTE:** Executar ações especiais (relatórios, etc.)

7.4 Tabela: PerfilPermissao (Associativa)

Para que serve: Relaciona perfis com suas permissões.

Exemplo prático:

Plain Text

Perfil "Gerente de Projetos" tem:

- CRM_PROJETO_CREATE
- CRM_PROJETO_READ
- CRM_PROJETO_UPDATE
- CRM_PROJETO_DELETE
- CRM_TAREFA_CREATE
- CRM_TAREFA_READ
- CRM_TAREFA_UPDATE
- CRM_TAREFA_DELETE
- CRM_PESSOA_READ

Perfil "Usuário Padrão" tem:

- CRM_TAREFA_READ (só suas tarefas)
- CRM_TAREFA_UPDATE (só suas tarefas)

7.5 Tabela: UsuarioPerfil (Associativa)

Para que serve: Relaciona usuários com seus perfis.

Exemplo prático:

Plain Text

Usuário "joao.silva" tem:

- Perfil: "Gerente de Projetos"
- Data atribuição: 01/01/2025
- Data expiração: NULL (sem expiração)

Usuário "maria.santos" tem:

- Perfil: "Usuário Padrão"
- Data atribuição: 01/01/2025
- Data expiração: 31/12/2025 (temporário)

Por que data de expiração?

- Perfis temporários (estagiários, consultores)
- Acesso por projeto específico

- Facilita auditoria

8. Conceitos Técnicos Explicados

8.1 Chave Primária (PRIMARY KEY - PK)

O que é: Identificador único de cada registro na tabela.

Exemplo:

SQL

```
CREATE TABLE Pessoa (  
    id INT IDENTITY(1,1) PRIMARY KEY,  
    nome NVARCHAR(200)  
);
```

Como funciona:

- Cada pessoa tem um `id` único (1, 2, 3, 4...)
- NUNCA dois registros terão o mesmo `id`
- `IDENTITY(1,1)` significa: começa em 1, incrementa de 1 em 1
- Gerado automaticamente pelo banco

Por que é importante:

- Garante que cada registro é único
- Permite referenciar registros de outras tabelas
- Melhora performance de consultas

8.2 Chave Estrangeira (FOREIGN KEY - FK)

O que é: Campo que referencia a chave primária de outra tabela, criando relacionamento.

Exemplo:

SQL

```
CREATE TABLE Filial (  
    id INT IDENTITY(1,1) PRIMARY KEY,  
    coligada_id INT NOT NULL,  
    nome NVARCHAR(200),  
  
    CONSTRAINT FK_Filial_Coligada
```

```
FOREIGN KEY (coligada_id)
REFERENCES Coligada(id)
);
```

Como funciona:

Plain Text

Coligada:

```
id=1, nome="Empresa Matriz"
id=2, nome="Empresa Filial SP"
```

Filial:

```
id=1, coligada_id=1, nome="Matriz São Paulo"
id=2, coligada_id=1, nome="Filial Rio"
id=3, coligada_id=2, nome="Filial SP Centro"
```

Regras:

- `coligada_id` na Filial DEVE existir na Coligada
- Não pode inserir Filial com `coligada_id=999` se não existe Coligada com `id=999`
- Garante integridade dos dados

8.3 ON DELETE e ON UPDATE

O que são: Definem o que acontece quando um registro pai é excluído ou atualizado.

ON DELETE CASCADE

Significado: Quando o pai é excluído, os filhos também são excluídos automaticamente.

Exemplo:

SQL

```
CONSTRAINT FK_Contato_Organizacao
FOREIGN KEY (organizacao_id)
REFERENCES Organizacao(id)
ON DELETE CASCADE
```

Como funciona:

Plain Text

```
Organização id=1 "Bevap"
├─ Contato id=1 "Maria"
```


└ Contato id=2 "Pedro"

Se você DELETAR a Organização id=1:

└ Os Contatos id=1 e id=2 serão DELETADOS automaticamente

Quando usar:

- Tabelas associativas (N:M)
- Registros dependentes que não fazem sentido sozinhos
- Exemplo: Contatos de uma empresa, itens de um orçamento

ON DELETE RESTRICT

Significado: NÃO permite excluir o pai se existirem filhos.

Exemplo:

SQL

```
CONSTRAINT FK_Filial_Coligada
  FOREIGN KEY (coligada_id)
  REFERENCES Coligada(id)
  ON DELETE RESTRICT
```

Como funciona:

Plain Text

Coligada id=1 "Empresa Matriz"

└ Filial id=1 "Matriz SP"

Se você tentar DELETAR a Coligada id=1:

└ ERRO! Não pode deletar porque existe Filial vinculada

└ Você DEVE deletar a Filial primeiro

Quando usar:

- Registros importantes que não devem ser excluídos acidentalmente
- Exemplo: Coligada com Filiais, Status em uso, Produto em orçamentos

ON DELETE SET NULL

Significado: Quando o pai é excluído, o campo FK nos filhos vira NULL.

Exemplo:

SQL

```
CONSTRAINT FK_Tarefa_Predecessora
  FOREIGN KEY (tarefa_predecessora_id)
  REFERENCES Tarefa(id)
  ON DELETE SET NULL
```

Como funciona:

Plain Text

Tarefa id=1 "Criar banco de dados"
Tarefa id=2 "Criar API" (predecessora_id=1)

Se você DELETAR a Tarefa id=1:
└ Tarefa id=2 fica com predecessora_id=NULL
└ A Tarefa id=2 NÃO é deletada

Quando usar:

- Relacionamentos opcionais
- Quando a exclusão do pai não deve afetar o filho
- Exemplo: Tarefa predecessora, responsável opcional

ON UPDATE CASCADE

Significado: Quando o ID do pai é atualizado, os FKs dos filhos são atualizados automaticamente.

Exemplo:

SQL

```
CONSTRAINT FK_Filial_Coligada
  FOREIGN KEY (coligada_id)
  REFERENCES Coligada(id)
  ON UPDATE CASCADE
```

Como funciona:

Plain Text

Coligada id=1 "Empresa Matriz"
└ Filial id=1, coligada_id=1

Se você ATUALIZAR Coligada id de 1 para 100:

└ Filial id=1 automaticamente fica com coligada_id=100

Quando usar:

- Quase sempre! É raro mudar IDs, mas CASCADE garante consistência
- Usado em praticamente todas as FKs

8.4 UNIQUE Constraint

O que é: Garante que um campo (ou combinação de campos) seja único na tabela.

Exemplo 1 - Campo único:

SQL

```
CREATE TABLE Pessoa (  
    id INT PRIMARY KEY,  
    cpf NVARCHAR(14) UNIQUE,  
    email NVARCHAR(100) UNIQUE  
);
```

Como funciona:

- Não pode ter duas pessoas com mesmo CPF
- Não pode ter duas pessoas com mesmo email
- NULL é permitido (a não ser que seja NOT NULL)

Exemplo 2 - Combinação única:

SQL

```
CREATE TABLE Status (  
    id INT PRIMARY KEY,  
    codigo NVARCHAR(50),  
    contexto NVARCHAR(20),  
  
    CONSTRAINT UQ_Status_Codigo_Contexto  
        UNIQUE (codigo, contexto)  
);
```

Como funciona:

Plain Text

Permitido:

- codigo="PENDENTE", contexto="TAREFA"
- codigo="PENDENTE", contexto="PROJETO"

NÃO permitido:

- codigo="PENDENTE", contexto="TAREFA" (duplicado!)

8.5 CHECK Constraint

O que é: Valida se o valor inserido atende uma condição.

Exemplo 1 - Valores permitidos:

SQL

```
CREATE TABLE Projeto (  
    id INT PRIMARY KEY,  
    prioridade NVARCHAR(20) CHECK (  
        prioridade IN ('BAIXA', 'MEDIA', 'ALTA', 'URGENTE')  
    )  
);
```

Como funciona:

- Só aceita: BAIXA, MEDIA, ALTA ou URGENTE
- Se tentar inserir "ALTISSIMA" → ERRO!

Exemplo 2 - Validação de datas:

SQL

```
CREATE TABLE Projeto (  
    id INT PRIMARY KEY,  
    data_inicio DATE,  
    data_fim_prevista DATE,  
  
    CONSTRAINT CHK_Projeto_Datas CHECK (  
        data_fim_prevista IS NULL OR  
        data_fim_prevista >= data_inicio  
    )  
);
```

Como funciona:

- Data fim DEVE ser maior ou igual à data início

- Ou pode ser NULL (sem previsão)

Exemplo 3 - Validação de percentual:

SQL

```
CONSTRAINT CHK_Projeto_Percentual CHECK (  
    percentual_conclusao >= 0 AND  
    percentual_conclusao <= 100  
)
```

8.6 DEFAULT

O que é: Define um valor padrão se nenhum for informado.

Exemplo:

SQL

```
CREATE TABLE Pessoa (  
    id INT PRIMARY KEY,  
    nome NVARCHAR(200),  
    ativo BIT DEFAULT 1,  
    created_at DATETIME2 DEFAULT GETDATE()  
);
```

Como funciona:

SQL

```
-- Inserindo sem informar ativo e created_at  
INSERT INTO Pessoa (nome) VALUES ('João Silva');  
  
-- Resultado:  
-- nome = 'João Silva'  
-- ativo = 1 (padrão)  
-- created_at = data/hora atual (padrão)
```

8.7 NOT NULL

O que é: Campo obrigatório, não pode ser vazio.

Exemplo:

SQL

```
CREATE TABLE Pessoa (  
  id INT PRIMARY KEY,  
  nome NVARCHAR(200) NOT NULL,  
  email NVARCHAR(100) NOT NULL,  
  telefone NVARCHAR(20) -- pode ser NULL  
);
```

Como funciona:

SQL

```
-- ERRO! nome é obrigatório  
INSERT INTO Pessoa (email) VALUES ('joao@email.com');  
  
-- OK! nome e email informados  
INSERT INTO Pessoa (nome, email)  
VALUES ('João', 'joao@email.com');
```

8.8 INDEX (Índice)

O que é: Estrutura que acelera consultas em campos específicos.

Analogia: Como o índice de um livro - você encontra rapidamente sem ler tudo.

Exemplo:

SQL

```
CREATE INDEX idx_pessoa_email ON Pessoa(email);  
CREATE INDEX idx_pessoa_cpf ON Pessoa(cpf);
```

Como funciona:

SEM índice:

SQL

```
SELECT * FROM Pessoa WHERE email = 'joao@email.com';  
-- Banco precisa ler TODAS as linhas (lento!)
```

COM índice:

SQL

```
SELECT * FROM Pessoa WHERE email = 'joao@email.com';  
-- Banco usa o índice e vai direto no registro (rápido!)
```

Quando criar índices:

- Chaves estrangeiras (sempre!)
- Campos usados em WHERE frequentemente
- Campos usados em JOIN
- Campos usados em ORDER BY

Cuidado:

- Índices ocupam espaço
- Índices deixam INSERT/UPDATE mais lentos
- Não crie índices em tudo!

8.9 Índice Composto

O que é: Índice em múltiplas colunas juntas.

Exemplo:

SQL

```
CREATE INDEX idx_projeto_coligada_status  
ON Projeto(coligada_id, status_id);
```

Quando usar:

SQL

```
-- Esta query se beneficia do índice composto  
SELECT * FROM Projeto  
WHERE coligada_id = 1 AND status_id = 5;  
  
-- Esta também (usa primeira coluna)  
SELECT * FROM Projeto WHERE coligada_id = 1;  
  
-- Esta NÃO se beneficia (não usa primeira coluna)  
SELECT * FROM Projeto WHERE status_id = 5;
```

Regra: Ordem das colunas importa! Coloque a mais filtrada primeiro.

8.10 Soft Delete (Exclusão Lógica)

O que é: Não deleta fisicamente, apenas marca como deletado.

Implementação:

SQL

```
CREATE TABLE Pessoa (  
    id INT PRIMARY KEY,  
    nome NVARCHAR(200),  
    deleted_at DATETIME2 NULL  
);
```

Como funciona:

Exclusão tradicional (hard delete):

SQL

```
DELETE FROM Pessoa WHERE id = 1;  
-- Registro SUMIU do banco!
```

Exclusão lógica (soft delete):

SQL

```
UPDATE Pessoa  
SET deleted_at = GETDATE()  
WHERE id = 1;  
  
-- Registro ainda existe, mas marcado como deletado
```

Consultas:

SQL

```
-- Apenas registros ativos  
SELECT * FROM Pessoa WHERE deleted_at IS NULL;  
  
-- Apenas registros deletados  
SELECT * FROM Pessoa WHERE deleted_at IS NOT NULL;  
  
-- Todos (incluindo deletados)  
SELECT * FROM Pessoa;
```

Vantagens:

- Mantém histórico
- Pode "desfazer" exclusão
- Auditoria completa
- Relatórios históricos

Desvantagens:

- Ocupa mais espaço
- Queries precisam filtrar `deleted_at IS NULL`
- Pode confundir se esquecer o filtro

8.11 Campos de Auditoria

O que são: Campos para rastrear quem criou/modificou e quando.

Implementação padrão:

SQL

```
CREATE TABLE Pessoa (  
    id INT PRIMARY KEY,  
    nome NVARCHAR(200),  
  
    -- Campos de auditoria  
    created_at DATETIME2 DEFAULT GETDATE(),  
    updated_at DATETIME2 DEFAULT GETDATE(),  
    created_by INT,  
    updated_by INT,  
    deleted_at DATETIME2 NULL,  
    ativo BIT DEFAULT 1  
);
```

Como funciona:

Ao criar:

SQL

```
INSERT INTO Pessoa (nome, created_by)  
VALUES ('João Silva', 1);  
  
-- Resultado:  
-- created_at = data/hora atual (automático)  
-- created_by = 1 (usuário que criou)
```

```
-- updated_at = data/hora atual (automático)
-- updated_by = NULL
```

Ao atualizar:

SQL

```
UPDATE Pessoa
SET nome = 'João Silva Santos',
    updated_by = 2
WHERE id = 1;

-- updated_at precisa ser atualizado manualmente
-- ou via trigger (automático)
```

Benefícios:

- Rastreabilidade completa
- Auditoria de mudanças
- Compliance (LGPD, SOX, etc.)
- Debugging (quem quebrou?)

9. Relacionamentos entre Tabelas

9.1 Relacionamento 1:N (Um para Muitos)

Exemplo: Uma Coligada tem várias Filiais.

Implementação:

SQL

```
CREATE TABLE Coligada (
    id INT PRIMARY KEY,
    nome NVARCHAR(200)
);

CREATE TABLE Filial (
    id INT PRIMARY KEY,
    coligada_id INT NOT NULL,
    nome NVARCHAR(200),
```

```
FOREIGN KEY (coligada_id) REFERENCES Coligada(id)
);
```

Visualização:

Plain Text

```
Coligada 1 "Empresa Matriz"
├─ Filial 1 "Matriz SP"
├─ Filial 2 "Filial RJ"
└─ Filial 3 "Filial BH"

Coligada 2 "Empresa Filial SP"
└─ Filial 4 "Filial SP Centro"
```

Características:

- FK está na tabela "muitos" (Filial)
- Uma Coligada → várias Filiais
- Uma Filial → uma Coligada

9.2 Relacionamento N:M (Muitos para Muitos)

Exemplo: Uma Pessoa tem várias Habilidades, uma Habilidade é de várias Pessoas.

Implementação (ERRADA - não funciona):

SQL

```
-- NÃO DÁ PARA FAZER ASSIM!
CREATE TABLE Pessoa (
    id INT PRIMARY KEY,
    habilidade_id INT -- qual habilidade? E se tiver várias?
);
```

Implementação (CORRETA - tabela associativa):

SQL

```
CREATE TABLE Pessoa (
    id INT PRIMARY KEY,
    nome NVARCHAR(200)
);

CREATE TABLE Habilidade (
    id INT PRIMARY KEY,
```

```

    nome NVARCHAR(100)
);

-- Tabela associativa (N:M)
CREATE TABLE PessoaHabilidade (
    id INT PRIMARY KEY,
    pessoa_id INT NOT NULL,
    habilidade_id INT NOT NULL,
    nivel NVARCHAR(20),

    FOREIGN KEY (pessoa_id) REFERENCES Pessoa(id),
    FOREIGN KEY (habilidade_id) REFERENCES Habilidade(id),
    UNIQUE (pessoa_id, habilidade_id)
);

```

Visualização:

Plain Text

```

Pessoa 1 "João"
├─ Habilidade 1 "Python" (AVANÇADO)
├─ Habilidade 2 "JavaScript" (INTERMEDIÁRIO)
└─ Habilidade 3 "Liderança" (AVANÇADO)

Pessoa 2 "Maria"
├─ Habilidade 1 "Python" (ESPECIALISTA)
└─ Habilidade 4 "Design" (AVANÇADO)

Habilidade 1 "Python"
├─ Pessoa 1 "João" (AVANÇADO)
└─ Pessoa 2 "Maria" (ESPECIALISTA)

```

Características:

- Precisa de tabela associativa (terceira tabela)
- Pode ter campos adicionais (nivel, data, etc.)
- UNIQUE (pessoa_id, habilidade_id) evita duplicação

9.3 Relacionamento 1:1 (Um para Um)

Exemplo: Uma Pessoa tem um Usuário.

Implementação:

SQL

```
CREATE TABLE Pessoa (
    id INT PRIMARY KEY,
    nome NVARCHAR(200)
);

CREATE TABLE Usuario (
    id INT PRIMARY KEY,
    pessoa_id INT NOT NULL UNIQUE, -- UNIQUE garante 1:1
    username NVARCHAR(50),

    FOREIGN KEY (pessoa_id) REFERENCES Pessoa(id)
);
```

Visualização:

Plain Text

```
Pessoa 1 "João" → Usuario 1 "joao.silva"
Pessoa 2 "Maria" → Usuario 2 "maria.santos"
Pessoa 3 "Pedro" → (sem usuário)
```

Características:

- FK com UNIQUE garante 1:1
- Nem toda Pessoa precisa ter Usuário
- Um Usuário sempre tem uma Pessoa

10. Exemplos Práticos de Uso

10.1 Cenário Completo: Do Orçamento ao Projeto

Passo 1: Cadastrar estrutura organizacional

SQL

```
-- Criar coligada
INSERT INTO Coligada (codigo, nome, cnpj)
VALUES ('COL001', 'Minha Empresa Ltda', '12.345.678/0001-90');

-- Criar filial
INSERT INTO Filial (coligada_id, codigo, nome, matriz)
VALUES (1, 'FIL-SP', 'Matriz São Paulo', 1);

-- Criar cliente
```

```
INSERT INTO Organizacao (tipo, codigo, nome_fantasia, cnpj_cpf, coligada_id)
VALUES ('CLIENTE', 'CLI001', 'Bevap Tecnologia', '98.765.432/0001-10', 1);
```

Passo 2: Cadastrar pessoas

SQL

```
-- Criar pessoa (funcionário)
INSERT INTO Pessoa (codigo, nome, email, tipo, coligada_id)
VALUES ('PES001', 'João Silva', 'joao@empresa.com', 'FUNCIONARIO', 1);

-- Criar usuário para a pessoa
INSERT INTO Usuario (pessoa_id, username, email, senha_hash)
VALUES (1, 'joao.silva', 'joao@empresa.com', '$2a$10$...');
```

Passo 3: Criar orçamento

SQL

```
-- Criar orçamento
INSERT INTO Orcamento (
    numero, organizacao_id, data_emissao, data_validade,
    titulo, valor_total, status_id, coligada_id, responsavel_id
) VALUES (
    'ORC-2025-001', 1, '2025-01-01', '2025-01-31',
    'Desenvolvimento Sistema Web', 50000.00, 1, 1, 1
);

-- Adicionar itens ao orçamento
INSERT INTO OrcamentoItem (
    orcamento_id, tipo, servico_id, descricao,
    quantidade, unidade, valor_unitario, valor_total
) VALUES
(1, 'SERVICO', 1, 'Desenvolvimento Backend', 200, 'Hora', 150.00, 30000.00),
(1, 'SERVICO', 2, 'Desenvolvimento Frontend', 120, 'Hora', 120.00, 14400.00);
```

Passo 4: Criar proposta

SQL

```
INSERT INTO Proposta (
    numero, orcamento_id, data_emissao, data_validade,
    titulo, valor_total, status_id, responsavel_id
) VALUES (
    'PROP-2025-001', 1, '2025-01-05', '2025-02-05',
```

```
'Proposta Sistema Web Bevap', 48000.00, 1, 1
);
```

Passo 5: Criar contrato

SQL

```
INSERT INTO Contrato (
    numero, proposta_id, dataassinatura, data_inicio,
    titulo, valor_total, status_id, responsavel_id
) VALUES (
    'CONT-2025-001', 1, '2025-01-10', '2025-01-15',
    'Contrato Sistema Web Bevap', 48000.00, 1, 1
);
```

Passo 6: Criar projeto

SQL

```
INSERT INTO Projeto (
    codigo, nome, organizacao_id, contrato_id,
    data_inicio, data_fim_prevista, valor_orcado,
    status_id, coligada_id, gerente_id
) VALUES (
    'PROJ-2025-001', 'Sistema Web Bevap', 1, 1,
    '2025-01-15', '2025-04-15', 48000.00,
    1, 1, 1
);
```

Passo 7: Adicionar fases ao projeto

SQL

```
INSERT INTO ProjetoFase (projeto_id, fase_id, ordem, status_id)
VALUES
(1, 1, 1, 1), -- Iniciação
(1, 2, 2, 1), -- Planejamento
(1, 3, 3, 1), -- Execução
(1, 5, 4, 1); -- Encerramento
```

Passo 8: Criar atividades

SQL

```
INSERT INTO Atividade (
    projeto_id, codigo, nome, horas_estimadas, status_id
```

```
) VALUES
(1, 'AT-001', 'Desenvolvimento Backend', 200, 1),
(1, 'AT-002', 'Desenvolvimento Frontend', 120, 1);
```

Passo 9: Criar tarefas

SQL

```
INSERT INTO Tarefa (
    atividade_id, codigo, titulo, horas_estimadas,
    prioridade, status_id
) VALUES
(1, 'TAR-001', 'Criar modelo de dados', 20, 'ALTA', 1),
(1, 'TAR-002', 'Implementar API autenticação', 30, 'ALTA', 1);
```

Passo 10: Atribuir pessoas às tarefas

SQL

```
INSERT INTO TarefaPessoa (
    tarefa_id, pessoa_id, papel, horas_estimadas, data_atribuicao
) VALUES
(1, 1, 'RESPONSAVEL', 20, '2025-01-15'),
(2, 1, 'RESPONSAVEL', 30, '2025-01-20');
```

10.2 Consultas Úteis

Listar projetos de um cliente:

SQL

```
SELECT
    p.codigo,
    p.nome,
    o.nome_fantasia AS cliente,
    p.data_inicio,
    p.percentual_conclusao,
    s.nome AS status,
    pes.nome AS gerente
FROM Projeto p
JOIN Organizacao o ON p.organizacao_id = o.id
JOIN Status s ON p.status_id = s.id
JOIN Pessoa pes ON p.gerente_id = pes.id
WHERE o.id = 1;
```


Listar tarefas de uma pessoa:

SQL

```
SELECT
    t.codigo,
    t.titulo,
    a.nome AS atividade,
    proj.nome AS projeto,
    tp.papel,
    t.horas_estimadas,
    t.horas_realizadas,
    s.nome AS status
FROM Tarefa t
JOIN TarefaPessoa tp ON t.id = tp.tarefa_id
JOIN Atividade a ON t.atividade_id = a.id
JOIN Projeto proj ON a.projeto_id = proj.id
JOIN Status s ON t.status_id = s.id
WHERE tp.pessoa_id = 1
AND t.deleted_at IS NULL;
```

Calcular total de horas por projeto:

SQL

```
SELECT
    p.codigo,
    p.nome,
    SUM(t.horas_estimadas) AS total_horas_estimadas,
    SUM(t.horas_realizadas) AS total_horas_realizadas,
    (SUM(t.horas_realizadas) * 100.0 / NULLIF(SUM(t.horas_estimadas), 0)) AS
percentual_realizado
FROM Projeto p
JOIN Atividade a ON p.id = a.projeto_id
JOIN Tarefa t ON a.id = t.atividade_id
WHERE p.id = 1
GROUP BY p.codigo, p.nome;
```

Listar orçamentos pendentes:

SQL

```
SELECT
    orc.numero,
    o.nome_fantasia AS cliente,
    orc.data_emissao,
    orc.data_validade,
```

```
orc.valor_total,  
s.nome AS status,  
pes.nome AS responsavel  
FROM Orcamento orc  
JOIN Organizacao o ON orc.organizacao_id = o.id  
JOIN Status s ON orc.status_id = s.id  
JOIN Pessoa pes ON orc.responsavel_id = pes.id  
WHERE s.final = 0 -- Status não finais  
AND orc.deleted_at IS NULL  
ORDER BY orc.data_validade;
```



Resumo Final

Este banco de dados foi projetado para:

✓ **Gerenciar estrutura organizacional** (coligadas, filiais, clientes) ✓ **Controlar pessoas e recursos** (funcionários, habilidades, funções) ✓ **Automatizar fluxo comercial** (orçamento → proposta → contrato) ✓ **Gerenciar projetos completos** (fases, atividades, tarefas) ✓ **Controlar acesso e segurança** (usuários, perfis, permissões) ✓ **Manter histórico e auditoria** (soft delete, campos de auditoria) ✓ **Garantir integridade** (PKs, FKs, constraints) ✓ **Otimizar performance** (índices estratégicos)

Principais diferenciais:

- 🔒 Integridade referencial completa
- 📊 Rastreabilidade total (auditoria)
- 🚀 Performance otimizada (índices)
- ↺ Soft delete (mantém histórico)
- 🎯 Relacionamentos bem definidos
- 🛡️ Segurança granular (perfis/permissões)
- 📈 Escalável e expansível

Autor: Manus AI **Data:** 06 de Outubro de 2025 **Versão:** 1.0